JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util

Interface Spliterator.OfInt

All Superinterfaces:

Spliterator<Integer>, Spliterator.OfPrimitive<Integer,IntConsumer,Spliterator.OfInt>

All Known Implementing Classes:

Spliterators.AbstractIntSpliterator

Enclosing interface:

Spliterator<T>

```
public static interface Spliterator.OfInt
extends Spliterator.OfPrimitive<Integer,IntConsumer,Spliterator.OfInt>
```

A Spliterator specialized for `int` values.

Since:

1.8

Nested Class Summary

Nested classes/interfaces inherited from interface java.util.Spliterator

Spliterator.OfDouble, Spliterator.OfInt, Spliterator.OfLong, Spliterator.OfPrimitive<T,T_CONS,T_SPLITR extends Spliterator.OfPrimitive<T,T_CONS,T_SPLITR>>

Field Summary

Fields inherited from interface java.util.Spliterator

CONCURRENT, DISTINCT, IMMUTABLE, NONNULL, ORDERED, SIZED, SORTED, SUBSIZED

Method Summary

Abstract Methods | Default Methods

Modifier and Type	Method	Description
default void	forEachRemaining(Consumer<? super Integer> action)	Performs the given action for each remaining element, sequentially in the current thread, until all elements have been processed or the action throws an exception.
default void	forEachRemaining(IntConsumer action)	Performs the given action for each remaining element, sequentially in the current thread, until all elements have been processed or the

		action throws an exception.
default boolean	**tryAdvance**(**Consumer**<? super **Integer**> action)	If a remaining element exists, performs the given action on it, returning `true`; else returns `false`.
boolean	**tryAdvance**(**IntConsumer** action)	If a remaining element exists, performs the given action on it, returning `true`; else returns `false`.
Spliterator.OfInt	**trySplit**()	If this spliterator can be partitioned, returns a Spliterator covering elements, that will, upon return from this method, not be covered by this Spliterator.

Methods inherited from interface java.util.Spliterator

characteristics, estimateSize, getComparator, getExactSizeIfKnown, hasCharacteristics

Method Detail

trySplit

```
Spliterator.OfInt trySplit()
```

Description copied from interface: Spliterator

If this spliterator can be partitioned, returns a Spliterator covering elements, that will, upon return from this method, not be covered by this Spliterator.

If this Spliterator is Spliterator.ORDERED, the returned Spliterator must cover a strict prefix of the elements.

Unless this Spliterator covers an infinite number of elements, repeated calls to `trySplit()` must eventually return `null`. Upon non-null return:

- the value reported for `estimateSize()` before splitting, must, after splitting, be greater than or equal to `estimateSize()` for this and the returned Spliterator; and
- if this Spliterator is `SUBSIZED`, then `estimateSize()` for this spliterator before splitting must be equal to the sum of `estimateSize()` for this and the returned Spliterator after splitting.

This method may return `null` for any reason, including emptiness, inability to split after traversal has commenced, data structure constraints, and efficiency considerations.

Specified by:

`trySplit` in interface `Spliterator<Integer>`

Specified by:

`trySplit` in interface `Spliterator.OfPrimitive<Integer,IntConsumer,Spliterator.OfInt>`

Returns:

a Spliterator covering some portion of the elements, or null if this spliterator cannot be split

tryAdvance

```
boolean tryAdvance(IntConsumer action)
```

Description copied from interface: Spliterator.OfPrimitive

If a remaining element exists, performs the given action on it, returning `true`; else returns `false`. If this Spliterator is `Spliterator.ORDERED` the action is performed on the next element in encounter order. Exceptions thrown by the action are relayed to the caller.

Specified by:

`tryAdvance` in
interface `Spliterator.OfPrimitive<Integer,IntConsumer,Spliterator.OfInt>`

Parameters:

`action - The action`

Returns:

`false if no remaining elements existed upon entry to this method, else true.`

forEachRemaining

```
default void forEachRemaining(IntConsumer action)
```

Description copied from interface: Spliterator.OfPrimitive

Performs the given action for each remaining element, sequentially in the current thread, until all elements have been processed or the action throws an exception. If this Spliterator is `Spliterator.ORDERED`, actions are performed in encounter order. Exceptions thrown by the action are relayed to the caller.

Specified by:

`forEachRemaining` in
interface `Spliterator.OfPrimitive<Integer,IntConsumer,Spliterator.OfInt>`

Parameters:

`action - The action`

tryAdvance

```
default boolean tryAdvance(Consumer<? super Integer> action)
```

If a remaining element exists, performs the given action on it, returning `true`; else returns `false`. If this Spliterator is `Spliterator.ORDERED` the action is performed on the next element in encounter order. Exceptions thrown by the action are relayed to the caller.

Specified by:

`tryAdvance in interface Spliterator<Integer>`

Implementation Requirements:

If the action is an instance of IntConsumer then it is cast to IntConsumer and passed to tryAdvance(java.util.function.IntConsumer); otherwise the action is adapted to an instance of IntConsumer, by boxing the argument of IntConsumer, and then passed to tryAdvance(java.util.function.IntConsumer).

Parameters:

action - The action

Returns:

false if no remaining elements existed upon entry to this method, else true.

forEachRemaining

```
default void forEachRemaining(Consumer<? super Integer> action)
```

Performs the given action for each remaining element, sequentially in the current thread, until all elements have been processed or the action throws an exception. If this Spliterator is Spliterator.ORDERED, actions are performed in encounter order. Exceptions thrown by the action are relayed to the caller.

Specified by:

forEachRemaining in interface Spliterator<Integer>

Implementation Requirements:

If the action is an instance of IntConsumer then it is cast to IntConsumer and passed to forEachRemaining(java.util.function.IntConsumer); otherwise the action is adapted to an instance of IntConsumer, by boxing the argument of IntConsumer, and then passed to forEachRemaining(java.util.function.IntConsumer).

Parameters:

```
action - The action
```


Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.